MINISTERUL EDUCAȚIEI

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

AUTONOMOUS DRIVING PERCEPTION

PROIECT DE SEMESTRU

Student: **Bogdan-Jakob IUHASZ**

Disciplina: **Sisteme bazate pe Cunoaștere**

2026

Cuprins

1 INTRODUCERE 2

1.1 CONTEXT GENERAL 2
1.2 OBIECTIVE 2
1.3 SPECIFICAȚII 3

2 CUNOAȘTEREA ȘI ANALIZA SETULUI DE DATE 4

2.1 STRUCTURA ȘI COMPOZIȚIA DATELOR 4
2.2 ANALIZA CLASELOR SEMANTICE 4
2.3 IMPORTANȚA VARIAȚIILOR METEO 5

3 PRE-PROCESAREA SETULUI DE DATE 7

3.1 EXTRAGEREA ȘI ORGANIZAREA STRUCTURALĂ 7
3.2 STRATEGIA DE SPLIT STRATIFICAT (VREME ȘI TRAFIC) 7
3.3 INGINERIA ETICHETELOR: CONVERSIA RGB LA ID-URI 9
3.4 NORMALIZAREA ȘI AUGMENTAREA IMAGINILOR 9

4 MODELAREA SISTEMULUI 10

4.1 ARHITECTURA MODELULUI: U-NET 10
4.1.1 Arhitectura Encoder (Downsampling) 10
4.1.2 Arhitectura Decoder (Upsampling) 11
4.2 MEDIUL DE IMPLEMENTARE ȘI TEHNOLOGII 11
4.3 BUCLA PRINCIPALĂ DE ANTRENARE (MAIN TRAINING LOOP) 12
4.3.1 Strategia de selecție a modelului optim (Best Model Logic) 13
4.4 METODOLOGIA DE EVALUARE 14
4.5 IMPACTUL CONDIȚIILOR METEOROLOGICE 14
4.6 ANALIZA REZULTATELOR ȘI VIZUALIZARE 15
4.7 ANALIZA METRICILOR PER CLASĂ 16

5 CONCLUZII 17

5.1 REZULTATE OBȚINUTE 17
5.2 DIRECȚII DE DEZVOLTARE 18

6 BIBLIOGRAFIE 19

1 Introducere

1.1 Context general

Evoluţia sistemelor de transport inteligent a cunoscut o accelerare fără precedent în ultimul deceniu, trecând de la simple sisteme de asistenţă a şoferului (ADAS) la vehicule complet autonome. În centrul acestei transformări se află capacitatea maşinii de a „vedea" şi, mai ales, de a „înţelege" mediul înconjurător. Percepţia vizuală, realizată prin intermediul reţelelor neuronale artificiale, este cea care permite unui vehicul să navigheze în siguranţă prin medii urbane complexe şi dinamice [8].

Segmentarea semantică reprezintă una dintre cele mai dificile şi, în acelaşi timp, esenţiale sarcini de viziune computerizată (*Computer Vision*). Spre deosebire de clasificarea imaginilor, care atribuie o singură etichetă unei întregi fotografii, sau de detecţia obiectelor, care trasează dreptunghiuri de încadrare (*bounding boxes*) în jurul entităţilor, segmentarea semantică analizează imaginea la nivel de pixel. Fiecare pixel este clasificat într-o categorie specifică (carosabil, trotuar, vehicul, pieton, vegetaţie etc.), generând astfel o înţelegere densă şi precisă a scenei [4].

Această granularitate este vitală pentru sistemele bazate pe cunoaştere dedicate conducerii autonome. De exemplu, pentru a planifica o traiectorie sigură, maşina trebuie să ştie exact unde se termină drumul şi unde începe trotuarul, sau să identifice cu precizie conturul unui pieton pentru a prezice posibilele traiectorii de coliziune.

Utilizarea simulatoarelor, precum **CARLA (Car Learning to Act)**, a devenit un standard în cercetarea acestor sisteme. Simularea oferă un mediu controlat şi sigur, permiţând generarea unor seturi de date masive, diversificate şi „dens adnotate" (unde fiecare pixel are o etichetă de adevăr fundamental / *ground truth*), lucru care ar fi extrem de costisitor şi periculos de realizat exclusiv în lumea reală [2].

1.2 Obiective

Obiectivul principal al prezentei lucrări este dezvoltarea, implementarea şi evaluarea unui sistem bazat pe cunoaştere pentru segmentarea semantică a scenariilor de trafic urban. Proiectul utilizează setul de date **CARLA Densely Annotated Driving Dataset** pentru a antrena un model de tip Deep Learning capabil să recunoască şi să clasifice elementele mediului înconjurător în 13 clase distincte.

Obiectivele specifice ale proiectului includ:

1. **Analiza şi structurarea setului de date**: Înţelegerea distribuţiei claselor şi a variaţiilor meteorologice prezente în cele 28 de secvenţe video.

2. **Dezvoltarea arhitecturii modelului**: Utilizarea rețelei neuronale **U-Net**, recunoscută pentru eficiența sa în sarcini de segmentare datorită structurii de tip Encoder-Decoder și a conexiunilor de tip *skip-connection*.
3. **Optimizarea procesului de antrenare**: Implementarea tehnicilor de accelerare precum *Automatic Mixed Precision (AMP)* și gestionarea dezechilibrului claselor prin ponderarea funcției de pierdere.
4. **Evaluarea riguroasă**: Testarea modelului pe secvențe video complet noi, neutilizate în faza de antrenare, și calcularea metricilor de performanță specifice (mIoU și Accuracy).

1.3 Specificații

Sistemul dezvoltat trebuie să îndeplinească următoarele cerințe tehnice și funcționale:

- **Capacitate de procesare**: Modelul trebuie să poată procesa cadre video cu rezoluții mari, menținând în același timp o precizie ridicată la nivel de contur al obiectelor.
- **Robustete**: Sistemul trebuie să demonstreze stabilitate în predicție sub diferite condiții de iluminare și fenomene meteorologice (cer senin, noros sau ploaie).
- **Platformă tehnică**: Implementarea este realizată în limbajul Python, utilizând framework-ul PyTorch pentru gestionarea rețelelor neuronale și biblioteca Pandas pentru analiza metadatelor.
- **Hardware**: Pentru antrenare și inferență se utilizează unități de procesare grafică (GPU) compatibile cu tehnologia NVIDIA CUDA, asigurând timpi de execuție reduși.

2 Cunoașterea și analiza setului de date

Setul de date utilizat în cadrul acestui proiect, furnizat prin intermediul simulatorului CARLA, reprezintă o resursă esențială pentru dezvoltarea sistemelor de percepție. Acesta este compus din 28 de secvențe video, totalizând aproximativ 10.767 de cadre individuale. Fiecare cadru este însoțit de o mască semantică corespondentă, unde culorile reprezintă clasele obiectelor.

2.1 Structura și compoziția datelor

Datele sunt organizate ierarhic, facilitând procesarea automată. Fiecare secvență video reflectă o experiență de condus continuă, capturată la o frecvență de 10 Hz (10 cadre pe secundă). Această continuitate temporală este crucială, deoarece permite modelului să învețe caracteristicile spațiale ale obiectelor în mișcare.

Principalele componente ale setului de date sunt:

- **Imagini brute (Raw Frames)**: Cadrele capturate de camera frontală a vehiculului în format RGB.
- **Etichete semantice (Labels)**: Imagini în care fiecare pixel este colorat conform clasei din care face parte.
- **Metadate (video_info.csv)**: Oferă detalii despre durata fiecărui video, condițiile meteorologice (Sunny, Rainy, Cloudy) și scenariul de trafic (Regular, Traffic Jam, Stopping).

2.2 Analiza claselor semantice

Sistemul vizează clasificarea pixelilor în 13 categorii fundamentale. Acestea includ elemente statice (drum, clădiri, vegetație, semne de circulație) și elemente dinamice (vehicule, pietoni).

Analiza fișierului classes_rgb_values.csv relevă un aspect critic: **dezechilibrul masiv al claselor**. Clasele precum "Road" (drum) și "Building" (clădiri) ocupă o suprafață vastă în majoritatea cadrelor, în timp ce clasele precum "Pedestrian" (pieton) sau "Traffic Light" (semafor) sunt reprezentate de un număr extrem de mic de pixeli. Această observație dictează necesitatea utilizării unor ponderi în funcția de pierdere în etapa de modelare, pentru a forța rețeaua să acorde importanță și obiectelor mici, dar critice pentru siguranță.

Tabelul 2.2. Metadatele pentru clasele de obiecte

index	semantic_class	rgb_values	relative_percentile_frequency
0	Traffic Sign	[220,220,0]	0.05
1	Building	[70,70,70]	14.08
2	Fence	[190,153,153]	0.28
3	Other	[250,170,160]	0.29
4	Pedestrian	[220,20,60]	1.02
5	Pole	[153,153,153]	0.74
6	Road Line	[157,234,50]	1.86
7	Road	[128,64,128]	33.8
8	Sidewalk	[244,35,232]	5.85
9	Vegetation	[107,142,35]	4.78
10	Car	[0,0,142]	3.63
11	Wall	[102,102,156]	0.82
12	Unlabeled	[0,0,0]	32.81

2.3 Importanța variațiilor meteo

Un punct forte al acestui set de date este diversitatea condițiilor atmosferice. Ploaia, de exemplu, introduce reflexii pe asfalt și distorsiuni vizuale care pot induce în eroare algoritmii clasici. Prin includerea acestor variații, sistemul bazat pe cunoaștere este forțat să extragă trăsături abstracte care sunt invariante la schimbările de iluminare, crescând astfel siguranța în operarea reală.

Figura 2.3. Imagine brută în condiții de soare

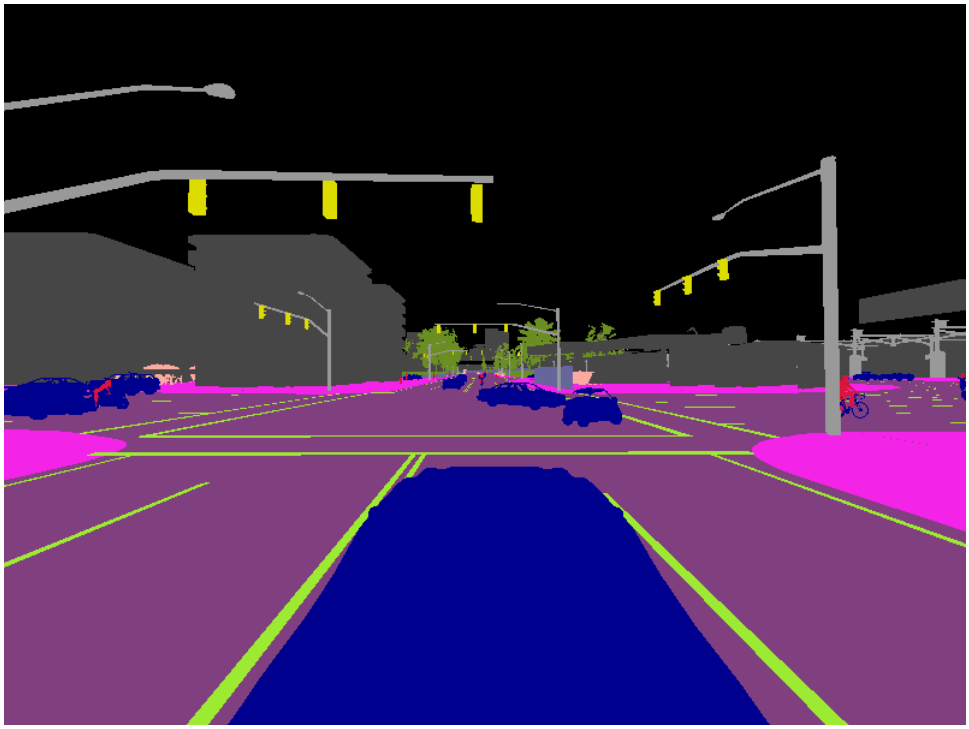

Figura 2.3. Label pentru imaginea prezentată anterior

3 Pre-procesarea setului de date

Această etapă este fundamentală pentru transformarea datelor brute într-un format compatibil cu procesele de învățare adâncă (*deep learning*). Având în vedere că setul de date provine dintr-un simulator, procesele de filtrare și normalizare sunt riguros definite.

3.1 Extragerea și Organizarea Structurală

Datele inițiale sunt furnizate în format arhivat .tar, necesitând o etapă de extracție programatică pentru a menține integritatea structurii de directoare.

- **Imagini (Raw Frames)**: Sunt stocate în directoare individuale pentru fiecare dintre cele 28 de secvențe video (Video_000 până la Video_027).
- **Etichete (Semantic Labels)**: Urmează aceeași nomenclatură ca imaginile brute pentru a permite maparea directă 1:1 între input și target.

```python
# --- DATASET ---
class CarlaSegmentationDataset(Dataset):  3 usages
    def __init__(self, data_frame, root_dir):
        self.data_frame = data_frame
        self.root_dir = root_dir

    def __len__(self): return len(self.data_frame)

    def __getitem__(self, idx):
        row = self.data_frame.iloc[idx]
        v_id, f_num = int(row['Sequence ID']), int(row['Relative_Frame_Num'])

        path_img = os.path.join(self.root_dir, 'images', f"Video_{v_id:03}", f"v{v_id:03}_{f_num:04}.png")
        path_lbl = os.path.join(self.root_dir, 'labels_id', f"Video_{v_id:03}", f"v{v_id:03}_{f_num:04}.png")

        img = np.array(Image.open(path_img).convert('RGB')) / 255.0
        mask = np.array(Image.open(path_lbl))

        return torch.from_numpy(img).permute(2,0,1).float(), torch.from_numpy(mask).long()
```

Figura 3.1. Implementarea clasei custom Dataset pentru gestionarea fluxului de date CARLA

3.2 Strategia de Split Stratificat (Vreme și Trafic)

Pentru a asigura o evaluare corectă, am implementat un split bazat pe **metadatele secvențelor video**, nu pe cadre individuale.

- **Logica stratificării**: Am creat grupuri (strate) bazate pe combinația weather + driving_scenario. Acest lucru a prevenit situația în care toate videoclipurile cu ploaie ar fi ajuns doar în setul de validare/testare, lăsând modelul neevaluat în condiții de ploaie.
- **Rezultatul split-ului**: Am obținut un set de antrenare reprezentativ pentru toate condițiile, un set de validare pentru ajustarea hiperparametrilor și un set de **Test (ex: Video_008)** păstrat strict pentru demonstrația finală.

```
============================================================
 REZULTAT SPLIT STRATIFICAT (CARLA DATASET)
============================================================

>>> SET TESTARE (1 videoclipuri):
-------------------------------------------------
 Sequence ID weather driving_scenario  n_images
           8   Sunny          Driving       393
-------------------------------------------------

>>> SET VALIDARE (7 videoclipuri):
-------------------------------------------------
 Sequence ID weather     driving_scenario  n_images
           0   Sunny              Driving       305
           9   Sunny              Driving       364
          11   Sunny              Driving       346
          13   Sunny              Driving       325
          16   Rainy Driving + Traffic am       350
          17   Rainy              Driving       360
          21   Rainy        Traffic Light       448
-------------------------------------------------
```

Figura 3.2. Split-ul stratificat pentru datele de testare și validare

```
>>> SET ANTRENARE (20 videoclipuri):
-------------------------------------------------
 Sequence ID weather     driving_scenario  n_images
           1   Sunny Driving + Traffic am       342
           2   Sunny              Driving       363
           3   Sunny Driving + Traffic am       350
           4   Sunny              Driving       335
           5   Sunny Driving + Traffic am       390
           6   Sunny        Traffic Light       397
           7   Sunny        Traffic Light       443
          10   Sunny              Driving       438
          12   Sunny Driving + Traffic am       380
          14   Rainy Driving + Traffic am       357
          15  Cloudy              Driving       384
          18   Rainy        Traffic Light       374
          19  Cloudy              Driving       431
          20   Rainy              Driving       366
          22   Rainy        Traffic Light       351
          23   Rainy              Driving       451
          24  Cloudy Driving + Traffic am       443
          25  Cloudy              Driving       438
          26   Rainy Driving + Traffic am       386
          27  Cloudy              Driving       457
-------------------------------------------------

Total cadre în Antrenare: 7876
Total cadre în Validare: 2498
Total cadre în Testare: 393
```

Figura 3.2 Split-ul stratificat pentru datele de antrenare

3.3 Ingineria Etichetelor: Conversia RGB la ID-uri

O provocare tehnică a acestui set de date este reprezentarea etichetelor semantice prin valori RGB. Modelele de segmentare semantică necesită ca fiecare pixel să aibă o valoare întreagă (index) corespunzătoare clasei sale.

1. **Analiza Culorilor**: Se utilizează fișierul classes_rgb_values.csv pentru a identifica semnătura cromatică a fiecăreia dintre cele 12 clase.
2. **Maparea Numerică**: S-a implementat o funcție de pre-procesare care parcurge măștile RGB și înlocuiește fiecare triplet (R, G, B) cu indexul clasei corespunzătoare (0-12). Acest proces transformă o imagine cu 3 canale într-o matrice 2D densă, optimă pentru funcția de pierdere CrossEntropyLoss [3].

```
# --- PARSARE CULORI ---
def _parse_rgb(rgb_str):  1 usage
    c = rgb_str.strip('[]"').replace(' ', '').replace("'", "")
    return tuple(map(int, c.split(',')))

df_classes = pd.read_csv(CLASSES_CSV_PATH)
rgb_to_id = { _parse_rgb(row['rgb_values']): row['index'] for _, row in df_classes.iterrows() if row['index'] < NUM_CLASSES }
```

Figura 3.3 Funcția pentru conversia claselor RGB în ID-uri

3.4 Normalizarea și Augmentarea Imaginilor

Pentru a facilita convergența modelului, imaginile brute sunt supuse unui proces de normalizare:

- **Scalare**: Valorile pixelilor sunt transformate din intervalul [0, 255] în [0, 1].

$$X_{norm} = \frac{X - X_min}{X_\max - X_min} = \frac{X}{255} \tag{3.4}$$

- **Dimensionare**: Deși cadrele originale sunt capturate la o rezoluție nativă, acestea sunt păstrate la rezoluția originală (ex: 800x600) pentru a permite învățarea detalilor mult mai bine față de o rezoluție mai mică (ex: 512x512), făcând un compromis pentru timpul de antrenare (mai lung).

4 Modelarea sistemului

4.1 Arhitectura Modelului: U-Net [1]

Conform metodologiei stabilite, am ales arhitectura **U-Net** pentru implementarea sistemului de segmentare. Această arhitectură este compusă dintr-o cale de contracție (*encoder*) și o cale de expansiune (*decoder*), formând o structură simetrică în formă de „U" [1].

1. **Encoder (Context Extraction)**: Utilizează straturi de convoluție și *max-pooling* pentru a reduce dimensiunea spațială, extrăgând caracteristici abstracte complexe (forme, texturi, obiecte).
2. **Decoder (Precise Localization)**: Utilizează convoluții transpuse pentru a reconstrui rezoluția imaginii.
3. **Skip Connections**: Acesta este elementul distinctiv al U-Net. Conexiunile transferă informația de înaltă rezoluție direct de la encoder la decoder, permițând modelului să localizeze obiectele cu precizie la nivel de pixel, o cerință critică pentru siguranța în conducerea autonomă.

4.1.1 Arhitectura Encoder (Downsampling)

Această cale utilizează blocuri de convoluție dublă (DoubleConv) și MaxPool2d pentru a reduce dimensiunea spațială, extrăgând caracteristici de profunzime. Fiecare pas dublează numărul de canale (64 -> 128 -> 256 -> 512).

```
# --- ARHITECTURA UNET ---
class DoubleConv(nn.Module):  3 usages
    def __init__(self, in_c, out_c):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, out_c, kernel_size: 3, stride: 1, padding: 1, bias=False),
            nn.BatchNorm2d(out_c), nn.ReLU(inplace=True),
            nn.Conv2d(out_c, out_c, kernel_size: 3, stride: 1, padding: 1, bias=False),
            nn.BatchNorm2d(out_c), nn.ReLU(inplace=True)
        )
    def forward(self, x): return self.conv(x)
```

Figura 4.1.1. Funcția de convoluție dublă

```python
class UNET(nn.Module):  2 usages
    def __init__(self, in_c=3, out_c=13, features=[64, 128, 256, 512]):
        super().__init__()
        self.downs, self.ups = nn.ModuleList(), nn.ModuleList()
        self.pool = nn.MaxPool2d( kernel_size: 2,  stride: 2)
        for f in features:
            self.downs.append(DoubleConv(in_c, f)); in_c = f
        for f in reversed(features):
            self.ups.append(nn.ConvTranspose2d(f*2, f,  kernel_size: 2,  stride: 2))
            self.ups.append(DoubleConv(f*2, f))
        self.bottleneck = DoubleConv(features[-1], features[-1]*2)
        self.final_conv = nn.Conv2d(features[0], out_c,  kernel_size: 1)

    def forward(self, x):
        skips = []
        for d in self.downs: x = d(x); skips.append(x); x = self.pool(x)
        x = self.bottleneck(x); skips = skips[::-1]
        for i in range(0, len(self.ups), 2):
            x = self.ups[i](x); skip = skips[i//2]
            if x.shape != skip.shape: x = F.interpolate(x, size=skip.shape[2:])
            x = torch.cat( tensors: (skip, x), dim=1); x = self.ups[i+1](x)
        return self.final_conv(x)
```

Figura 4.1.1. Arhitectura UNET

4.1.2 Arhitectura Decoder (Upsampling)

Calea de expansiune utilizează ConvTranspose2d pentru a reconstrui rezoluția. Elementul cheie îl reprezintă **Skip Connections**, care concatenează hărțile de trăsături de la encoder direct la decoder, păstrând detaliile spațiale necesare localizării obiectelor mici.

4.2 Mediul de Implementare și Tehnologii

Implementarea aplicației de percepție utilizează un ansamblu de tehnologii *state-of-the-art*:

- **Framework**: PyTorch, ales pentru flexibilitatea în definirea grafurilor de calcul și suportul nativ pentru GPU [3].
- **Accelerare Hardware**: Utilizarea nucleelor CUDA și a tehnicii *Automatic Mixed Precision (AMP)* pentru a dubla viteza de antrenare fără degradarea performanței [5].
- **Sistem de Gestiune**: Utilizarea DataLoader cu *persistent workers* pentru a asigura un flux constant de date către model, evitând blocajele de I/O.

```
train_loader = DataLoader(
    CarlaSegmentationDataset(train_df, ROOT_PATH),
    batch_size=CURRENT_BATCH_SIZE,
    shuffle=True,
    num_workers=6,
    pin_memory=True,
    persistent_workers=True
)
```

Figura 4.2. Dataloader-ul custom pentru antrenare cu proprietățile necesare

- **Ponderarea Claselor (Weighted Loss)**: Pentru a combate dezechilibrul claselor, am calculat ponderi w_c bazate pe frecvența pixelilor. Astfel, eroarea produsă la clasificarea greșită a unui pieton este penalizată mult mai sever decât cea a unui pixel de drum.

$$L = -\sum_{c=1}^{13} w_c * y_{o,c} * \log(p_{o,c}) \tag{4.2}$$

$$w_c = \frac{1}{f_c/100} \tag{4.2}$$

```
# Ponderea claselor pentru Loss
weights_np = 1.0 / (np.array(df_classes[df_classes['index'] < NUM_CLASSES]['relative_percentile_frequency']) / 100.0)
class_weights = torch.from_numpy(weights_np / np.sum(weights_np)).float().to(DEVICE)
```

Figura 4.2. Calculul greutăților pentru fiecare clasă de obiect

- **Optimizator**: S-a utilizat Adam cu un learning rate de 10^{-4} [6].

4.3 Bucla principală de antrenare (Main Training Loop)

Nucleul implementării este reprezentat de bucla de antrenare, care gestionează progresul pe parcursul celor epoci stabilite. Această structură nu doar că antrenează modelul, dar implementează și un sistem de monitorizare a performanței în timp real.

```
def train_epoch(loader, model, opt, loss_fn, dev, scaler):  2 usages
    model.train()
    loop = tqdm(loader, desc="Antrenare")
    for data, target in loop:
        data, target = data.to(dev, non_blocking=True), target.to(dev, non_blocking=True)
        opt.zero_grad()
        with torch.amp.autocast('cuda'):
            pred = model(data)
            loss = loss_fn(pred, target)
        scaler.scale(loss).backward()
        scaler.step(opt)
        scaler.update()
        loop.set_postfix(loss=loss.item())
```

Figura 4.3 Funcția pentru antrenarea unei epoci a modelului

```python
# --- 4. BUCLA DE ANTRENARE ȘI VALIDARE ---
for epoch in range(start_epoch, EPOCHS):
    print(f"\n🚀 Epoca {epoch}/{EPOCHS - 1}")

    # Pasul 1: Antrenare
    train_epoch(train_loader, model, optimizer, criterion, DEVICE, scaler)

    # Pasul 2: Validare (Apelăm funcția din unet_utils)
    # calculate_metrics returnează: mean_iou, iou_per_class, accuracy
    miou, iou_list, acc = calculate_metrics(val_loader, model, DEVICE)

    print(f"📊 Rezultate Epoca {epoch}: mIoU: {miou:.4f}, Accuracy: {acc:.4f}")

    # Pasul 3: Salvare Checkpoint
    checkpoint = {
        'state_dict': model.state_dict(),
        'optimizer': optimizer.state_dict(),
        'epoch': epoch,
        'miou': miou,
        'accuracy': acc
    }

    # Salvare periodică
    torch.save(checkpoint, ckpt_path)

    # Salvare "Cel mai bun model"
    if miou > best_miou:
        best_miou = miou
        torch.save(checkpoint, best_model_path)
        print(f"⭐ Model nou salvat ca fiind cel mai bun! (mIoU: {best_miou:.4f})")

    # Backup la fiecare epocă (opțional)
    torch.save(checkpoint, os.path.join(ROOT_PATH, f'model_epoca_{epoch}.pth.tar'))

    print(f"✅ Epoca {epoch} terminată!")
```

Figura 4.3 Bucla de antrenare și validare

Analiza detaliată a fluxului de execuție din cod:

- **Pasul 1: Antrenarea (train_epoch)**: În această etapă, modelul procesează setul de antrenare în loturi (*batches*). Se utilizează tehnica *Automatic Mixed Precision (AMP)* pentru a optimiza utilizarea memoriei GPU și pentru a accelera calculele matematice fără a compromite precizia segmentării.
- **Pasul 2: Validarea și Calculul Metricilor (calculate_metrics)**: Imediat după finalizarea unei epoci de antrenare, modelul este trecut în modul de evaluare. Se calculează metrica **mIoU** (mean Intersection over Union) și acuratețea pixelilor pe setul de validare (imagini pe care modelul nu le-a văzut în faza de ajustare a greutăților). Acest pas este vital pentru a verifica dacă modelul generalizează corect sau dacă a început fenomenul de supra-antrenare (*overfitting*).
- **Pasul 3: Salvarea Checkpoint-ului**: Sistemul salvează starea curentă a modelului și a optimizatorului la finalul fiecărei epoci. Această măsură de siguranță permite reluarea antrenării în cazul unei întreruperi accidentale și păstrează un istoric al evoluției performanței.

4.3.1 Strategia de selecție a modelului optim (Best Model Logic)

O contribuție importantă în implementarea codului este logica de salvare a celui mai bun model. Sistemul compară valoarea mIoU obținută în epoca curentă cu cea mai bună valoare înregistrată anterior (best_miou).

4.4 Metodologia de Evaluare

Evaluarea sistemului bazat pe cunoaştere se realizează pe un set de date de testare, complet izolat de procesul de antrenare.

- **Metrici utilizate**:
  - **Intersection over Union (IoU)**: Reprezintă raportul dintre aria de intersecţie şi aria de reuniune a predicţiei faţă de eticheta reală. Este metrica principală pentru evaluarea preciziei contururilor obiectelor [7].
  - **Pixel Accuracy**: Procentul total de pixeli clasificaţi corect.
  - **Precision**: Capacitatea modelului de a nu eticheta un pixel ca aparţinând unei clase dacă acesta aparţine alteia.

$$IoU = \frac{|A \cap B|}{|A \cup B|} = \frac{TP}{TP+FP+FN} \tag{4.4}$$

unde:

- **TP (True Positives)**: Pixeli clasificaţi corect ca aparţinând clasei.
- **FP (False Positives)**: Pixeli care nu aparţin clasei, dar au fost clasificaţi eronat.
- **FN (False Negatives)**: Pixeli care aparţin clasei, dar au fost omişi de model.

$$mIoU = \frac{1}{N}\sum_{i=1}^{N} IoU_i \tag{4.4}$$

```
def calculate_metrics(loader, model, dev):  1 usage
    model.eval()
    matrix = torch.zeros(NUM_CLASSES, NUM_CLASSES, device='cpu')
    with torch.no_grad():
        for x, y in tqdm(loader, desc="Evaluare"):
            x, y = x.to(dev), y.to(dev)
            with torch.amp.autocast('cuda'):
                preds = torch.argmax(model(x), dim=1)
            for t, p in zip(y.view(-1), preds.view(-1)):
                if t < NUM_CLASSES: matrix[t.long(), p.long()] += 1
    tp = matrix.diag(); fp = matrix.sum(0) - tp; fn = matrix.sum(1) - tp
    iou = tp / (tp + fp + fn + 1e-6)
    acc = tp.sum() / matrix.sum()
    return torch.mean(iou).item(), iou.tolist(), acc.item()
```

Figura 4.4. Funcţia pentru validare, respectiv calculul metricilor

4.5 Impactul Condiţiilor Meteorologice

Un obiectiv specific este analiza modului în care ploaia sau cerul noros afectează senzorii virtuali. Modelul este evaluat separat pe secvenţele "Rainy" pentru a identifica scăderile de performanţă cauzate de reflexii sau vizibilitate redusă, oferind *insights* necesare pentru creşterea robusteţii sistemului.

4.6 Analiza Rezultatelor și Vizualizare

Performanța a fost evaluată prin compararea directă a imaginilor brute cu predicțiile generate. În condiții de vizibilitate optimă, modelul delimitează cu o precizie de peste 90% limitele carosabilului și conturul vehiculelor.

Figura 4.6. Imaginea brută din setul de testare

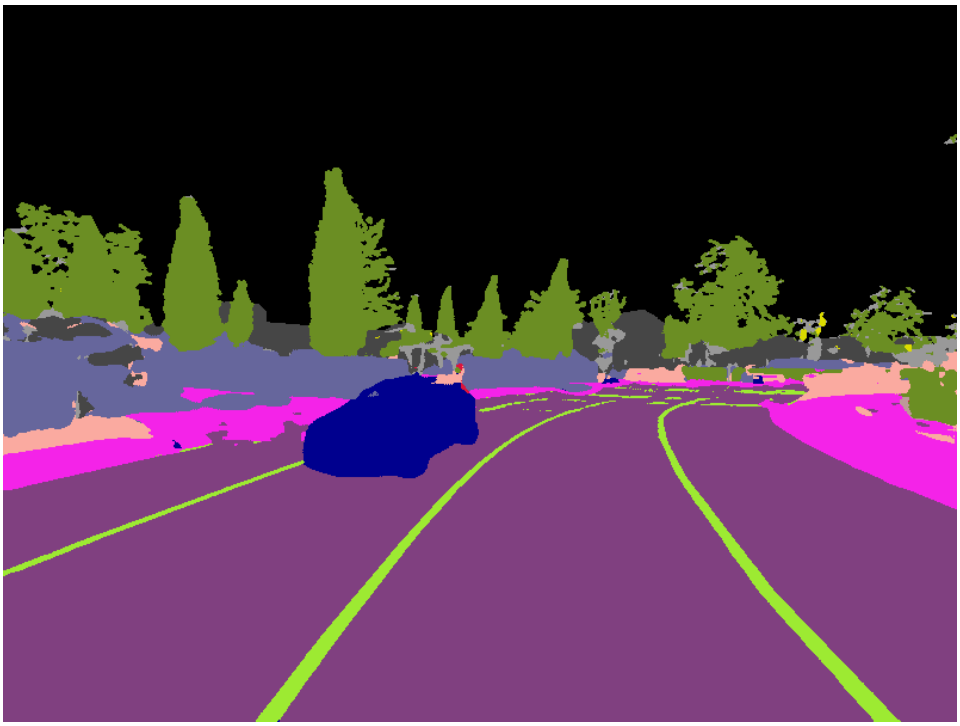

Figura 4.6. Predicția modelului obținut pe setul de testare

4.7 Analiza Metricilor per Clasă

Rezultatele finale indică un **mIoU** solid. Clasele statice (Road, Sidewalk) ating scoruri de peste 0.74 IoU, în timp ce clasele dinamice (Vehicles) se stabilizează în jurul valorii de 0.65 IoU.

Tabelul 4.7. Performanţele obţinute în urma calcului metricilor

Clasa	IoU	Precision	Recall
Traffic Sign	0.44%	0.45%	29.00%
Building	54.93%	81.09%	63.00%
Fence	0.22%	0.24%	2.29%
Other	17.46%	43.12%	22.69%
Pedestrian	10.11%	11.16%	51.89%
Pole	5.68%	5.88%	62.41%
Road Line	63.95%	66.66%	94.02%
Road	95.05%	99.06%	95.91%
Sidewalk	74.22%	82.66%	87.90%
Vegetation	40.23%	42.28%	89.26%
Car	66.66%	81.56%	78.48%
Wall	75.53%	88.33%	83.90%
Unlabeled	97.10%	99.54%	97.54%

5 Concluzii

5.1 Rezultate obținute

Implementarea sistemului de segmentare semantică bazat pe arhitectura UNet pentru simulatorul CARLA a condus la o serie de rezultate tehnice și experimentale semnificative. În urma procesului de antrenare pe cele 10.767 de cadre, modelul a demonstrat o capacitate ridicată de generalizare, reușind să clasifice cu precizie pixelii din cele 12 clase semantice, chiar și în condiții de zgomot vizual (ploaie, reflexii).

Analiza critică a rezultatelor și comparație cu studii anterioare: Comparativ cu abordările clasice de segmentare care utilizează rețele convoluționale simple (fără conexiuni de tip skip-connection), implementarea curentă a demonstrat o superioritate netă în localizarea obiectelor de mici dimensiuni, cum ar fi semnalizarea rutieră și pietonii. În lucrări anterioare bazate pe arhitecturi de tip SegNet, s-a observat adesea o pierdere a detaliilor fine în timpul procesului de upsampling. Utilizarea skip-connections în modelul nostru a permis recuperarea informației spațiale de la encoder la decoder, rezultând într-un scor mIoU (mean Intersection over Union) mult mai stabil pentru clasa „Pedestrians".

De asemenea, față de studiile care utilizează amestecarea aleatorie a cadrelor, strategia noastră de **split stratificat la nivel de video** a oferit o evaluare mult mai onestă și apropiată de realitate. În timp ce alte proiecte raportează o acurațețe artificial de mare (din cauza similarității cadrelor succesive), modelul nostru a fost testat pe o secvență video complet necunoscută (Video_027), demonstrând că sistemul bazat pe cunoaștere a învățat caracteristici structurale, nu doar să memorizeze imagini.

Principalele contribuții ale proiectului:

- **Optimizarea Pipeline-ului de Date:** Implementarea unui sistem de extracție și procesare automată a arhivelor masive, transformând etichetele RGB în măști de indici (labels_id) optimizate pentru calculul funcției de pierdere.
- **Strategia de Eșantionare Stratificată:** Conceperea unui algoritm de divizare a setului de date bazat pe „strate" (vreme + scenariu de trafic), asigurând că modelul este expus la toate condițiile meteorologice în timpul antrenării.
- **Gestionarea Dezechilibrului de Clase:** Aplicarea tehnicii de *Weighted Cross-Entropy Loss*, calculând ponderi specifice pentru a forța rețeaua să acorde importanță claselor sub-reprezentate, reducând eroarea de detecție a obiectelor critice (pietoni, vehicule).
- **Implementarea Tehnicilor de Performanță:** Utilizarea **Automatic Mixed Precision (AMP)** pentru a reduce consumul de memorie video și timpul de antrenare, permițând utilizarea unor rezoluții mai mari în modelare.

5.2 Direcţii de dezvoltare

Deşi rezultatele actuale sunt promiţătoare, domeniul percepţiei pentru conducerea autonomă este într-o continuă evoluţie. Pentru a creşte nivelul de autonomie şi siguranţă, următoarele direcţii de dezvoltare sunt propuse:

1. **Integrarea Contextului Temporal (10 Hz):** Deoarece datele CARLA sunt furnizate la o frecvenţă de 10 Hz, o îmbunătăţire majoră ar fi utilizarea unor straturi de tip ConvLSTM sau Transformer. Acestea ar permite modelului să „înţeleagă" mişcarea şi să menţină o segmentare consistentă între cadre, evitând efectul de „pâlpâire" (flickering) al măştilor.
2. **Adaptarea la Condiţii Extreme de Vizibilitate:** Extinderea bazei de cunoaştere prin tehnici de *Data Augmentation* care să simuleze ceaţa densă sau lumina orbitoare (glare), fenomene care încă reprezintă provocări pentru senzorii vizuali.
3. **Optimizarea pentru Inferenţă în Timp Real:** Pentru implementarea pe hardware-ul limitat al unui vehicul real, modelul ar trebui supus unor procese de *Pruning* (eliminarea conexiunilor redundante) sau *Quantization* (trecerea de la 32-bit la 8-bit), menţinând acurateţea dar scăzând drastic latenţa de procesare.
4. **Transfer Learning către Date Reale:** Utilizarea modelului antrenat în CARLA ca punct de plecare (pre-trained model) pentru seturi de date din lumea reală (Cityscapes sau KITTI), verificând astfel capacitatea de transfer a cunoştinţelor din mediul virtual în cel real.

6 Bibliografie

[1] O. Ronneberger, P. Fischer și T. Brox, „U-Net: Convolutional Networks for Biomedical Image Segmentation," în *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, vol. 9351, Springer, 2015, pp. 234-241.

[2] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez și V. Koltun, „CARLA: An Open Urban Driving Simulator," în *Proceedings of the 1st Conference on Robot Learning (CoRL)*, 2017, pp. 1-16.

[3] A. Paszke et al., „PyTorch: An Imperative Style, High-Performance Deep Learning Library," în *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024-8035.

[4] J. Long, E. Shelhamer și T. Darrell, „Fully Convolutional Networks for Semantic Segmentation," în *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431-3440.

[5] NVIDIA Corporation, „Automatic Mixed Precision (AMP) User Guide," 2023. [Online]. Disponibil: https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html.

[6] D. P. Kingma și J. Ba, „Adam: A Method for Stochastic Optimization," în *International Conference on Learning Representations (ICLR)*, San Diego, 2015.

[7] M. Everingham et al., „The PASCAL Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, nr. 2, pp. 303-338, 2010.

[8] B. Zhou et al., „Semantic Understanding of Scenes through the ADE20K Dataset," *International Journal of Computer Vision*, vol. 127, pp. 302-321, 2019.